

AI 221 Machine Exercise 5

Problem 1 - Country Data from HELP International

by Quiel Andrew I. Quiwa and Joel Paolo Salvador

In this machine exercise, we will analyze a dataset containing 167 countries with their respective development indicators on child mortality, exports, health, imports, income, life expectancy, fertility, and GDP.

We will be performing k-means clustering and hierarchical clustering to categorize the countries with respect to their development indicators. Through this, we will be able to see which countries are similar with each other with respect to level of development. We will also be able to see which indicators are causing the underdevelopment of the countries, which can serve as guidance on which types of interventions must be prioritized by countries to achieve higher levels of development.

Following the initial clustering methods, we will also be performing PCA to reduce the features data into 2D and then we will cluster the flattened data using K-means clustering again. This will be culminated by an anomaly detection method using isolation forest to identify countries that are outliers, so we can see which ones have either extremely positive or negative values for certain indicators.

```
In [17]: from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.pipeline import Pipeline
from sklearn.metrics import silhouette_score

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
In [18]: country_data = pd.read_csv('Country-data.csv')
country_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 167 entries, 0 to 166
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   country         167 non-null   object
1   child_mort      167 non-null   float64
2   exports         167 non-null   float64
3   health          167 non-null   float64
4   imports         167 non-null   float64
5   income          167 non-null   int64
6   inflation       167 non-null   float64
7   life_expec      167 non-null   float64
8   total_fer       167 non-null   float64
9   gdpp            167 non-null   int64
dtypes: float64(7), int64(2), object(1)
memory usage: 13.2+ KB
```

1. Identifying the Ideal Number of Clusters

```
In [59]: # Normalize the features data using Standard Scaler. Then, perform K-means clustering on all features.
# Display the elbow plot (Inertia vs. no. of clusters) and the silhouette score plot.

X = country_data.drop('country', axis=1)
X_scaled = StandardScaler().fit_transform(X)

# The inertia_values list is created to store the inertia values for each k (number of clusters) from 1 to 10.
# Inertia is a measure of how well the data points are clustered.
# It is calculated as the sum of squared distances of samples to their closest cluster center.
inertia_values = []

# The range from 1 to 11 is used to iterate through different values of k (number of clusters) for KMeans.
# n_init=10 means that the KMeans algorithm will be run 10 times with different centroid seeds, and the best output in terms
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertia_values.append(kmeans.inertia_)

# The average silhouette scores for each k is calculated and stored in silhouette_avgs list.
# The silhouette score is a measure of how similar an object is to its own cluster compared to other clusters.
# A higher silhouette score indicates that the object is well matched to its own cluster and poorly matched to neighboring cl
# The best k is determined by finding the maximum silhouette score, which indicates the optimal number of clusters for the da
silhouette_avgs = []

# The range from 2 to 11 is used to iterate through different values of k (number of clusters) for KMeans
# This is starting from 2 because silhouette score cannot be calculated for k=1.
for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    silhouette_avgs.append(silhouette_score(X_scaled, kmeans.labels_))

kmeans_results_inertia = pd.DataFrame({
    'K': range(1, 11),
    'Inertia': [round(v, 2) for v in inertia_values], # skip k=1
}).set_index('K')
```

```
kmeans_results_silhouette = pd.DataFrame({
    'K': range(2, 11),
    'Silhouette Score': [round(v, 4) for v in silhouette_avgs]
}).set_index('K')

kmeans_combined = pd.concat([kmeans_results_inertia, kmeans_results_silhouette], axis=1)
print(kmeans_combined)
```

	Inertia	Silhouette Score
K		
1	1503.00	NaN
2	1050.21	0.2874
3	831.42	0.2833
4	700.52	0.2960
5	620.16	0.2993
6	558.47	0.2287
7	495.81	0.2477
8	457.59	0.2388
9	427.80	0.2073
10	403.23	0.2022

```
In [54]: # Best k = where silhouette score is highest
best_k = np.argmax(silhouette_avgs) + 2 # +2 because silhouette starts at k=2

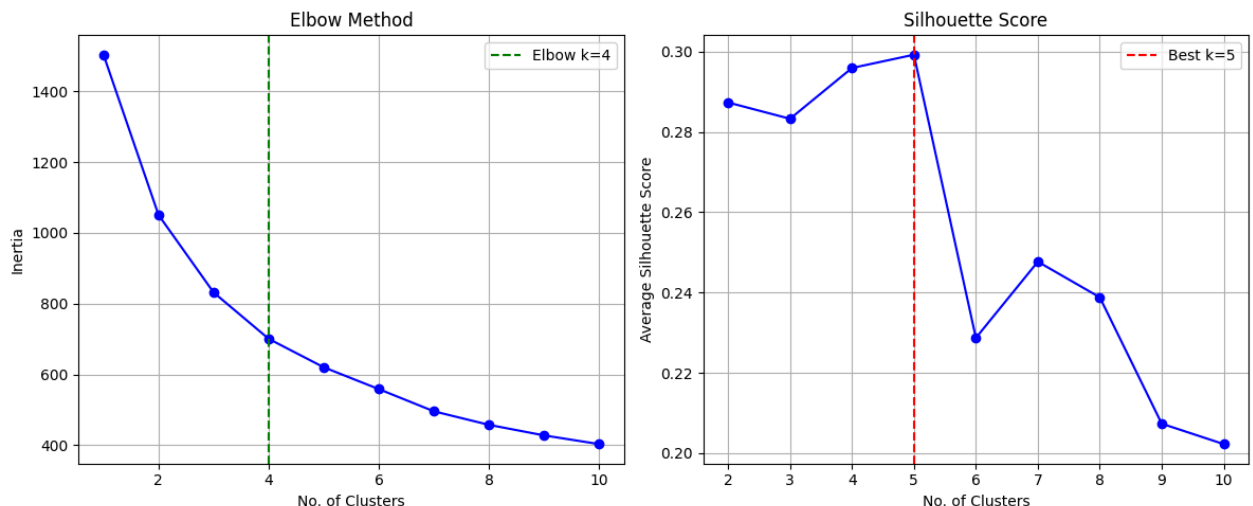
# Create a figure large enough to hold two subplots side by side
plt.figure(figsize=(12, 5))

# Create subplots for inertia and silhouette scores

# Inertia plot (Elbow Method)
# (1,2,1) subplot means 1 row, 2 columns, and this is the first plot.
plt.subplot(1,2,1)
plt.plot(np.arange(1, 11), inertia_values, 'o-b')
plt.axvline(x=4, color='green', linestyle='--', label=f'Elbow k={4}')
plt.grid()
plt.xlabel('No. of Clusters')
plt.ylabel('Inertia')
plt.title('Elbow Method')
plt.legend()

# Silhouette score plot
# (1,2,2) subplot means 1 row, 2 columns, and this is the second plot.
plt.subplot(1,2,2)
plt.plot(np.arange(2, 11), silhouette_avgs, 'o-b')
plt.axvline(x=best_k, color='red', linestyle='--', label=f'Best k={best_k}')
plt.grid()
plt.xlabel('No. of Clusters')
plt.ylabel('Average Silhouette Score')
plt.title('Silhouette Score')
plt.legend()

# Adjust layout and display the plots
plt.tight_layout()
plt.show()
```



Recommended Number of Clusters

The silhouette score peaks at $k=5$, and this is the recommended number of clusters. Visually, however, the elbow method shows $k=4$ to be farthest with respect to the perpendicular distance from an invisible line drawn from $k=1$ to $k=10$. While this is not the best k value with respect to the silhouette scores, we can still argue that $k=4$ is very close to the best k value. In fact, $k=3$, 4 , and 5 are very close in value based on their respective silhouette scores.

We will proceed with the hierarchical clustering using $k=5$ clusters since this is the clear peak k shown by the silhouette score.

2. Perform Hierarchical Clustering

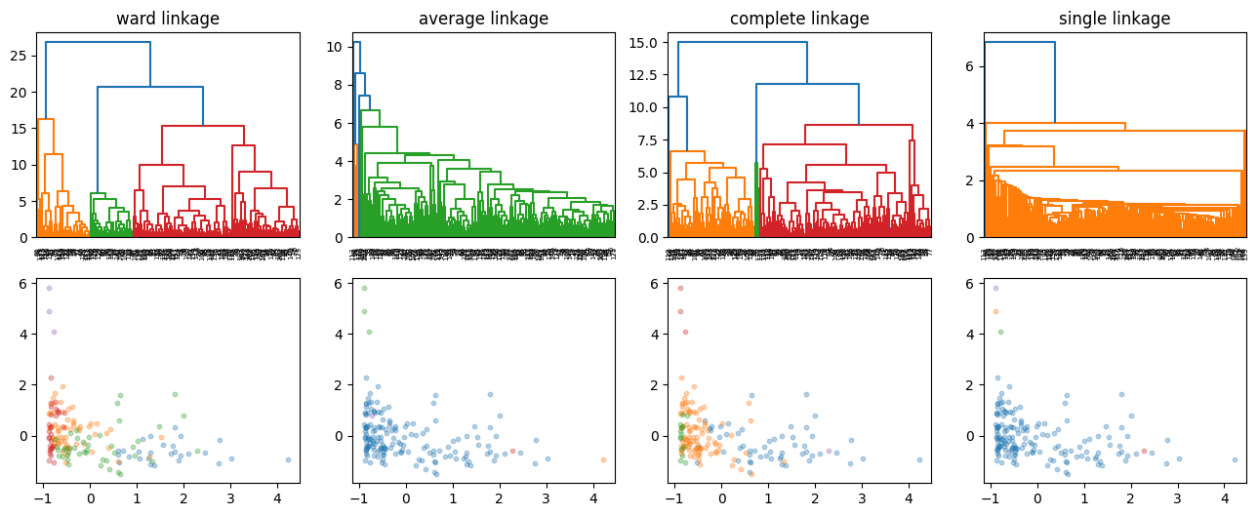
```
In [33]: from scipy.cluster.hierarchy import linkage, dendrogram, cut_tree
from time import time
```

```

fig3 = plt.figure(figsize=(16,6))
ctr = 1
t0 = time()
for method in ('ward', 'average', 'complete', 'single'):
    Z = linkage(X_scaled, method=method)
    ax = fig3.add_subplot(240 + ctr)
    ax.set_title("%s linkage" % method)
    dendrogram(Z)
    cutree = cut_tree(Z, n_clusters=5).flatten()
    ax = fig3.add_subplot(240 + ctr + 4)
    for j in range(0,5):
        ax.scatter(X_scaled[cutree == j,0], X_scaled[cutree == j,1], s=10, alpha=0.3)
    ctr += 1

plt.show()
print(f"Elapsed Time: {time()-t0} sec")

```



Elapsed Time: 2.133284091949463 sec

```

In [60]: hier_results = []
for method in ('ward', 'average', 'complete', 'single'):
    Z = linkage(X_scaled, method=method)
    cutree = cut_tree(Z, n_clusters=5).flatten()
    score = silhouette_score(X_scaled, cutree)
    sizes = sorted(np.bincount(cutree).tolist(), reverse=True)
    hier_results.append({
        'Linkage Method': method.capitalize(),
        'Silhouette Score': round(score, 4),
        'Cluster Sizes': str(sizes)
    })

hier_df = pd.DataFrame(hier_results).set_index('Linkage Method')
print(hier_df)

```

Linkage Method	Silhouette Score	Cluster Sizes
Ward	0.2193	[63, 43, 31, 27, 3]
Average	0.3858	[161, 3, 1, 1, 1]
Complete	0.2460	[95, 54, 14, 3, 1]
Single	0.4624	[163, 1, 1, 1, 1]

Recommended Hierarchical Clustering

Visually, we can see that Ward linkage has the most well-balanced cluster of sizes. Compared to average and single linkage where there's practically only one dominant cluster, and with complete linkage where two clusters are more dominant, which already gives us an idea that we can find distinct groups of countries, Ward linkage shows five clusters of sizes 63, 43, 31, 27, and 3. It groups countries into distinct segments that likely show their real-world socio-economic patterns.

The table of results that follow the plot figure looks interesting as well, though. We can see that Single and Average have the higher silhouette scores, while Ward has the lowest. However, we'll prioritize the balance and distribution of countries across clusters more than the silhouette score for interpretable groupings in this case.

Note that the silhouette score has no penalty for imbalanced or degenerate clusterings. It only measures how well each individual point fits, so clusters with only 1 sample automatically get a score of 1 and the score gets inflated. This is why looking at the cluster size is also important. The score is only usable when the clusters are well-balanced.

Given these considerations, we recommend using the Ward linkage since it does not produce any dominant cluster.

PCA + K-Means Clustering

Perform PCA to reduce the features data into 2D. Then, perform K-means clustering on the reduced data set just as in item (a).

Display the elbow plot and silhouette score plot as well. What number of clusters is recommended? You can label some countries in the 2D mapping.

```

In [64]: pca = PCA(n_components=2, random_state=0)
X_pca = pca.fit_transform(X_scaled)

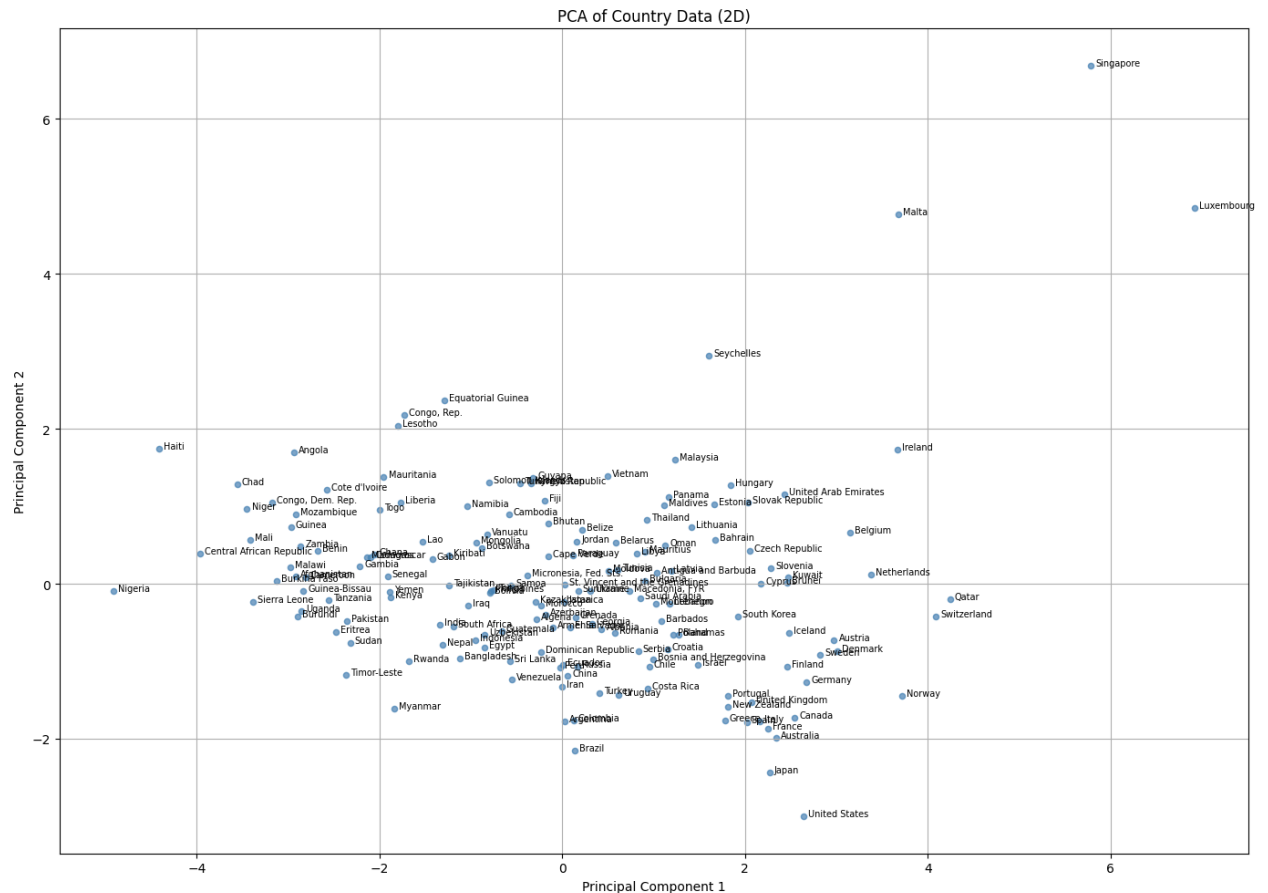
```

```

y = country_data['country']

plt.figure(figsize=(14, 10))
plt.scatter(X_pca[:, 0], X_pca[:, 1], color='steelblue', s=20, alpha=0.7)
for i in range(X_pca.shape[0]):
    plt.text(X_pca[i, 0] + 0.05, X_pca[i, 1], y[i], fontsize=7)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA of Country Data (2D)')
plt.grid()
plt.tight_layout()
plt.show()

```



```

In [65]: pca_inertia_values = []

# The range from 1 to 11 is used to iterate through different values of k (number of clusters) for KMeans.
# n_init=10 means that the KMeans algorithm will be run 10 times with different centroid seeds, and the best output in terms
for k in range(1, 11):
    pca_kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    pca_kmeans.fit(X_pca)
    pca_inertia_values.append(pca_kmeans.inertia_)

# The average silhouette scores for each k is calculated and stored in silhouette_avgs list.
# The silhouette score is a measure of how similar an object is to its own cluster compared to other clusters.
# A higher silhouette score indicates that the object is well matched to its own cluster and poorly matched to neighboring cl
# The best k is determined by finding the maximum silhouette score, which indicates the optimal number of clusters for the da
silhouette_avgs = []

# The range from 2 to 11 is used to iterate through different values of k (number of clusters) for KMeans
# This is starting from 2 because silhouette score cannot be calculated for k=1.
for k in range(2, 11):
    pca_kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    pca_kmeans.fit(X_pca)
    silhouette_avgs.append(silhouette_score(X_pca, pca_kmeans.labels_))

pca_kmeans_results_inertia = pd.DataFrame({
    'K': range(1, 11),
    'Inertia': [round(v, 2) for v in pca_inertia_values], # skip k=1
}).set_index('K')

pca_kmeans_results_silhouette = pd.DataFrame({
    'K': range(2, 11),
    'Silhouette Score': [round(v, 4) for v in silhouette_avgs]
}).set_index('K')

pca_kmeans_combined = pd.concat([pca_kmeans_results_inertia, pca_kmeans_results_silhouette], axis=1)
print(pca_kmeans_combined)

```

	Inertia	Silhouette Score
K		
1	948.89	NaN
2	503.02	0.4155
3	354.53	0.4416
4	246.70	0.3893
5	192.15	0.3821
6	156.73	0.3808
7	133.48	0.3627
8	115.94	0.3520
9	106.69	0.3330
10	91.43	0.3387

```
In [67]: # Best k = where silhouette score is highest
best_k_pca = np.argmax(silhouette_avgs) + 2 # +2 because silhouette starts at k=2

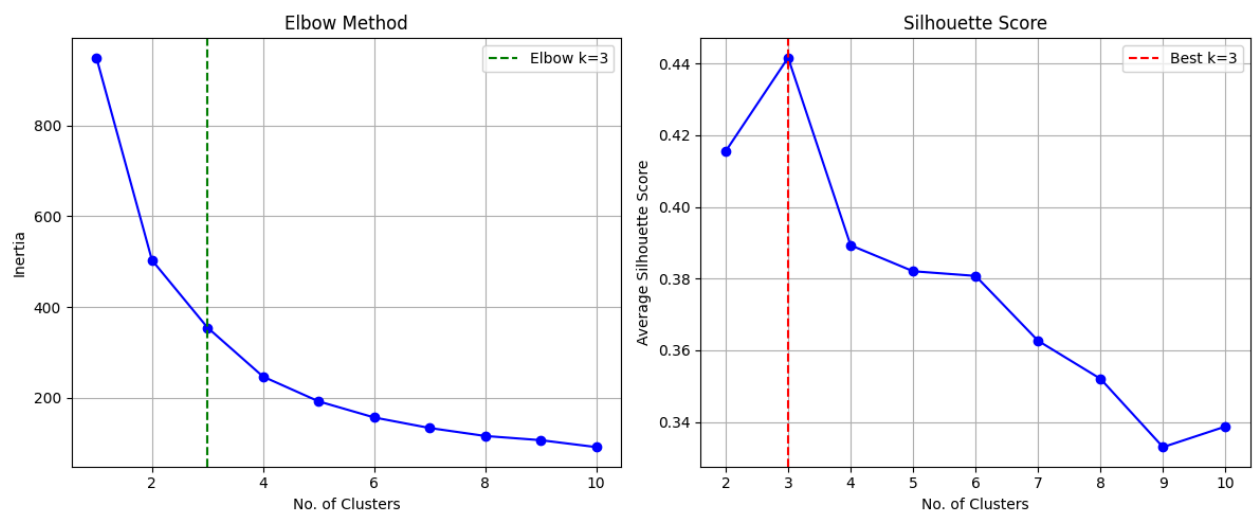
# Create a figure large enough to hold two subplots side by side
plt.figure(figsize=(12, 5))

# Create subplots for inertia and silhouette scores

# Inertia plot (Elbow Method)
# (1,2,1) subplot means 1 row, 2 columns, and this is the first plot.
plt.subplot(1,2,1)
plt.plot(np.arange(1, 11), pca_inertia_values, 'o-b')
plt.axvline(x=3, color='green', linestyle='--', label=f'Elbow k={3}')
plt.grid()
plt.xlabel('No. of Clusters')
plt.ylabel('Inertia')
plt.title('Elbow Method')
plt.legend()

# Silhouette score plot
# (1,2,2) subplot means 1 row, 2 columns, and this is the second plot.
plt.subplot(1,2,2)
plt.plot(np.arange(2, 11), silhouette_avgs, 'o-b')
plt.axvline(x=best_k_pca, color='red', linestyle='--', label=f'Best k={best_k_pca}')
plt.grid()
plt.xlabel('No. of Clusters')
plt.ylabel('Average Silhouette Score')
plt.title('Silhouette Score')
plt.legend()

# Adjust layout and display the plots
plt.tight_layout()
plt.show()
```



PCA Observations

After applying PCA, we recommend using $k=3$ clusters based on the "elbow" inertia value and the highest silhouette score. Visually, however, can observe from the 2D PCA graph that there are a couple of countries in the upper right (Singapore, Malta, Luxembourg, etc.) that might form their own small cluster. We will try to describe the clusters below.

Descriptions of Each Cluster

Based on the recommended no. of clusters in item (b), make your own descriptions of each cluster. What range of values of features are unique to each cluster?

```
In [73]: kmeans_pca = KMeans(n_clusters=3, random_state=42, n_init=10)
cluster_labels = kmeans_pca.fit_predict(X_pca)

plt.figure(figsize=(14, 10))
colors = ['#e41a1c', '#377eb8', '#4daf4a']
for cluster_id in range(3):
    mask = cluster_labels == cluster_id
    plt.scatter(X_pca[mask, 0], X_pca[mask, 1],
                color=colors[cluster_id], s=30, alpha=0.7,
```

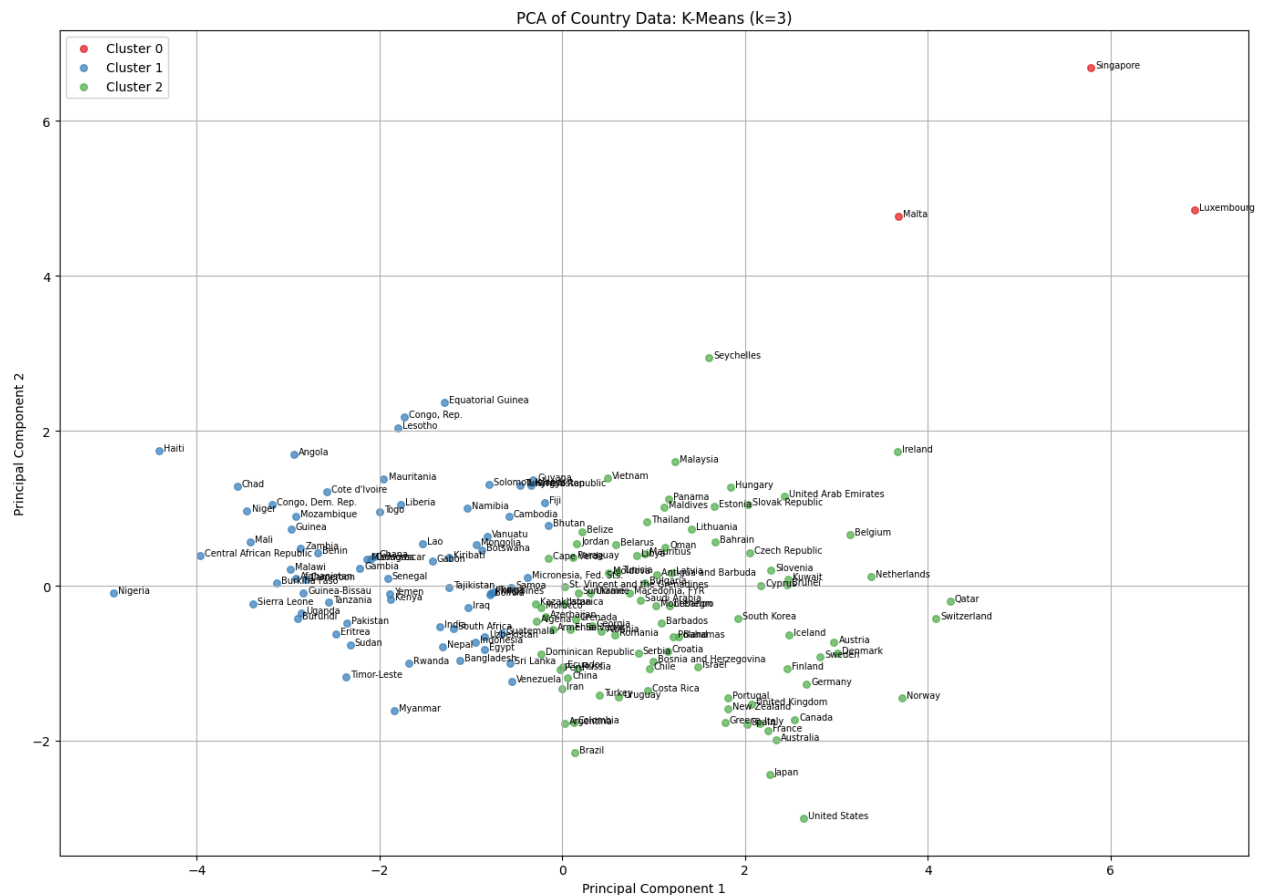
```

label=f'Cluster {cluster_id}'

for i, name in enumerate(country_data['country']):
    plt.text(X_pca[i, 0] + 0.05, X_pca[i, 1], name, fontsize=7)

plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA of Country Data: K-Means (k=3)')
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()

```



```

In [87]: loadings = pd.DataFrame(
    pca.components_.T,
    index=X.columns.tolist(),
    columns=['PC1', 'PC2'])
    .round(4)

loadings.sort_values('PC1', key=abs, ascending=False)

```

```

Out[87]:

```

	PC1	PC2
life_expec	0.4258	-0.2227
child_mort	-0.4195	0.1929
total_fer	-0.4037	0.1552
income	0.3984	0.0225
gdpp	0.3926	-0.0460
exports	0.2839	0.6132
inflation	-0.1932	-0.0084
imports	0.1615	0.6718
health	0.1508	-0.2431

```

In [88]: print(f"PC1 explains {pca.explained_variance_ratio_[0]*100:.1f}% of variance")
print(f"PC2 explains {pca.explained_variance_ratio_[1]*100:.1f}% of variance")
print(f"Total: {pca.explained_variance_ratio_.sum()*100:.1f}%")

```

PC1 explains 46.0% of variance
PC2 explains 17.2% of variance
Total: 63.1%

```

In [96]: kmeans_final = KMeans(n_clusters=best_k_pca, random_state=42, n_init=10)
labels = kmeans_final.fit_predict(X_pca)

cluster_df = X.copy() # original unscaled features
cluster_df['cluster'] = labels
cluster_df['country'] = country_data['country']

```

```
# Mean of each feature per cluster
print(cluster_df.groupby('cluster')[X.columns.tolist()].mean().round(2))
```

```

      child_mort  exports  health  imports   income  inflation \
cluster
0           4.13   176.00    6.79   156.67  64033.33      2.47
1           73.36    30.58    6.00    43.70   4531.85     11.58
2           11.92    44.95    7.45    45.80  25486.63      4.98

      life_expec  total_fer      gdpp
cluster
0          81.43      1.38  57566.67
1          62.46      4.30   2178.67
2          76.54      1.94  19950.54

```

```
In [84]: PH_index = country_data[country_data['country'] == 'Philippines'].index[0]
print(f"The Philippines is in Cluster {cluster_labels[PH_index]}")

countries_of_interest = ['Philippines', 'United States', 'Thailand', 'Vietnam', 'Malaysia']
comparison_df = cluster_df[cluster_df['country'].isin(countries_of_interest)].set_index('country')
comparison_df
```

The Philippines is in Cluster 1.

```
Out[84]:
```

	child_mort	exports	health	imports	income	inflation	life_expec	total_fer	gdpp	cluster
country										
Malaysia	7.9	86.9	4.39	71.0	21100	7.27	74.5	2.15	9070	2
Philippines	31.9	34.8	3.61	36.6	5600	4.22	69.0	3.16	2130	1
Thailand	14.9	66.5	3.88	60.8	13500	4.08	76.6	1.55	5080	2
United States	7.3	12.4	17.90	15.8	49400	1.22	78.7	1.93	48400	2
Vietnam	23.3	72.0	6.84	80.2	4490	12.10	73.1	1.95	1310	2

Describing each cluster using K=3

Visually, we can see that there are two major clusters formed by the 2D PCA decomposition, with a third smaller cluster (cluster 0) in the upper right segment.

Cluster 0 shows countries with low child mortality, high exports, high imports, high income, low inflation, longer life expectancy, and high GDP per capita. The values are extremely far from the values shown by clusters 1 and 2. Overall, we can say that these are countries that are doing really well with respect to the success indicators provided by this dataset.

Cluster 1 shows countries with high child mortality, high inflation, low income, low gdp, and high fertility rates. Exports, imports, health index, and life expectancy are also relatively lower compared to cluster 2. This could be a cluster formed to describe Developing countries. The Philippines belong to cluster 1, although some values in the Philippines' indexes are closer to cluster 2. We can use this information to check where the country should be spending resources to improve the overall living conditions of our citizens.

Cluster 2 shows countries from more Developed countries like the United States, Japan, and even ASEAN countries like Malaysia, Thailand, and Vietnam. It's frustrating to see how our ASEAN neighbors are doing so much better than us with generally better indexes across the board. Perhaps, it would also be valuable to look at other data like Typhoons and other disasters to check what kind of shocks our country is experiencing in comparison with our neighbors so that we can also spend more resources towards addressing those concerns.

PH Comparison

We can see that most values are generally better for the Philippines than for the average indexes of countries in cluster 1. However, we can also see that while we're doing better than a lot of developing countries in cluster 1, we're still far from the index values of countries in cluster 2. In fact, while inflation is shown here to be at 4.22, recent news from Bangko Sentral ng Pilipinas have shown that our inflation rates have ballooned to 7.2%. The Philippine Statistics Authority also reported in their Consumer Price Index that the bottom 30% of households feel this inflation rate even higher at 8.3%. We also have values in the dataset that are even lower than the average for cluster 1. GDP is lower at 2130. Health is worse at 3.61. This shows that interventions in health must be implemented since we're way behind even in comparison with developing countries.

PCA Features Loading

Based on the features loading table, we can see that PC1 explains 46% of the variance. This is heavily influenced by high life expectancy, low child mortality, low total fertility, high income, and high GDP.

Exports, imports, inflation, and health also affect PC1 but in lower magnitudes.

We can say that these indicators are the most impactful with respect to clustering countries from developing to developed countries.

Some countries with extraordinary circumstances like Singapore which has a relatively low population but performs well given these metrics are present, but these indicators give a fairly accurate clustering with respect to development indicators.

Anomaly Detection

```
In [94]: from sklearn.ensemble import IsolationForest

isoforest = IsolationForest(contamination=0.1, random_state=42)
outlier_labels = isoforest.fit_predict(X_pca)

outlier_countries = country_data['country'][outlier_labels == -1].values
print("Anomalous Countries Detected by Isolation Forest:")
```

```

print(outlier_countries)

plt.figure(figsize=(14, 10))
colors = ['red' if o == -1 else 'steelblue' for o in outlier_labels]
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=colors, s=30, alpha=0.7)
for i, name in enumerate(country_data['country']):
    if outlier_labels[i] == -1:
        plt.text(X_pca[i, 0] + 0.05, X_pca[i, 1], name, fontsize=8, color='red')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('Anomaly Detection on PCA (Isolation Forest)')
plt.grid()
plt.tight_layout()
plt.show()

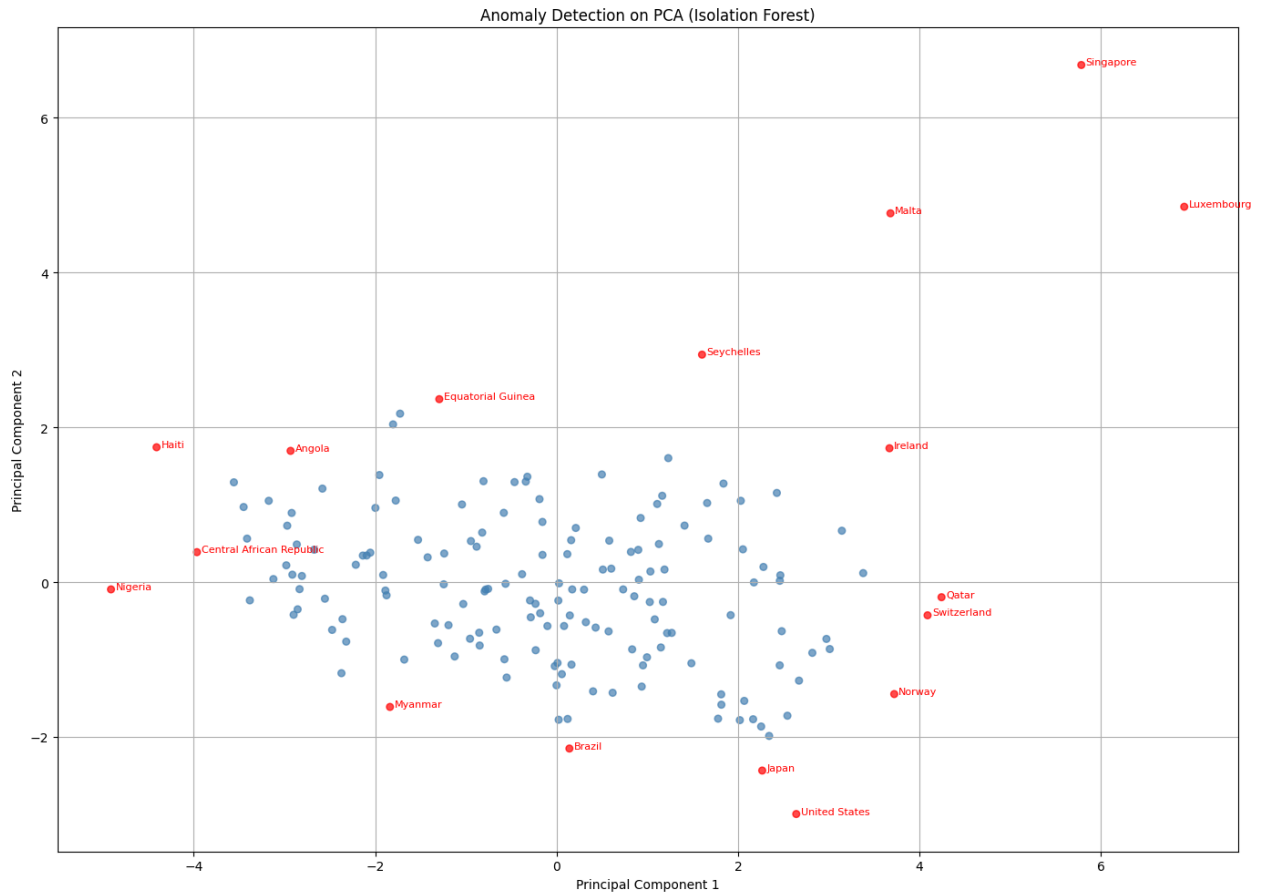
```

Anomalous Countries Detected by Isolation Forest:

```

['Angola' 'Brazil' 'Central African Republic' 'Equatorial Guinea' 'Haiti'
'Ireland' 'Japan' 'Luxembourg' 'Malta' 'Myanmar' 'Nigeria' 'Norway'
'Qatar' 'Seychelles' 'Singapore' 'Switzerland' 'United States']

```



```

In [95]: outlier_mask = outlier_labels == -1
print(cluster_df[outlier_mask][['country']] + X.columns.tolist())
print("\nDataset mean:\n", X.mean().round(2))

```


	country	child_mort	exports	health	imports	income \
3	Angola	119.0	62.300	2.85	42.9000	5900
22	Brazil	19.8	10.700	9.01	11.8000	14500
31	Central African Republic	149.0	11.800	3.98	26.5000	888
49	Equatorial Guinea	111.0	85.800	4.48	58.9000	33700
66	Haiti	208.0	15.300	6.91	64.7000	1500
73	Ireland	4.2	103.000	9.19	86.5000	45700
77	Japan	3.2	15.000	9.49	13.6000	35800
91	Luxembourg	2.8	175.000	7.77	142.0000	91700
98	Malta	6.8	153.000	8.65	154.0000	28300
107	Myanmar	64.4	0.109	1.97	0.0659	3720
113	Nigeria	130.0	25.300	5.07	17.4000	5150
114	Norway	3.2	39.700	9.48	28.5000	62300
123	Qatar	9.0	62.300	1.81	23.8000	125000
131	Seychelles	14.4	93.800	3.40	108.0000	20400
133	Singapore	2.8	200.000	3.96	174.0000	72100
145	Switzerland	4.5	64.000	11.50	53.3000	55500
159	United States	7.3	12.400	17.90	15.8000	49400

	inflation	life_expec	total_fer	gdpp
3	22.400	60.1	6.16	3530
22	8.410	74.2	1.80	11200
31	2.010	47.5	5.21	446
49	24.900	60.9	5.21	17100
66	5.450	32.1	3.33	662
73	-3.220	80.4	2.05	48700
77	-1.900	82.8	1.39	44500
91	3.620	81.3	1.63	105000
98	3.830	80.3	1.36	21100
107	7.040	66.8	2.41	988
113	104.000	60.5	5.84	2330
114	5.950	81.0	1.95	87800
123	6.980	79.5	2.07	70300
131	-4.210	73.4	2.17	10800
133	-0.046	82.7	1.15	46600
145	0.317	82.2	1.52	74600
159	1.220	78.7	1.93	48400

Dataset mean:

child_mort	38.27
exports	41.11
health	6.82
imports	46.89
income	17144.69
inflation	7.78
life_expec	70.56
total_fer	2.95
gdpp	12964.16

dtype: float64

Outlier countries

Each outlier country has either extremely high or low values for specific indicators. Looking at Singapore for example, it has extremely high exports (200) and extremely high GDP per capita (46600). Countries like Switzerland, Luxembourg, Norway, and Qatar also show similar high GDP per capita values. Countries like Luxembourg, Malta, and Ireland also show high exports similar to Singapore.

We have Nigeria with an inflation rate of 104, which is extremely high even in comparison with countries that already have high inflation rates like Angola, Congo, and Equatorial Guinea.

We have countries with high child mortality like Angola, Haiti, Equatorial Guinea, and Nigeria. Haiti even has an extremely low life expectancy at 32.1 years.

Negative inflation rates for countries like Ireland, Japan, Seychelles, and Singapore are notable.

Health is a major differentiator for outlier as well since we have countries like the United States and Switzerland which are way above the average shown in any of the clusters.

Overall, we can say that there are positive and negative factors that cause outlier detection among countries in the development map.